# Array-Based Sequences

Data Structures

Chapter 5.1-5.4, 5.6 in textbook

Based on textbook slides

# Python Sequence Classes

- Python has several built-in **sequence data types**, which are ordered collections of elements that can be indexed (using a syntax such as *A[i]*) to access individual items
- Most notable such types are
  - **str**
  - **tuple**
  - **list**
  - (also **bytes** and **bytearray)**
- Each of these types uses *(under the hood):*
  - an **array** to represent the sequence

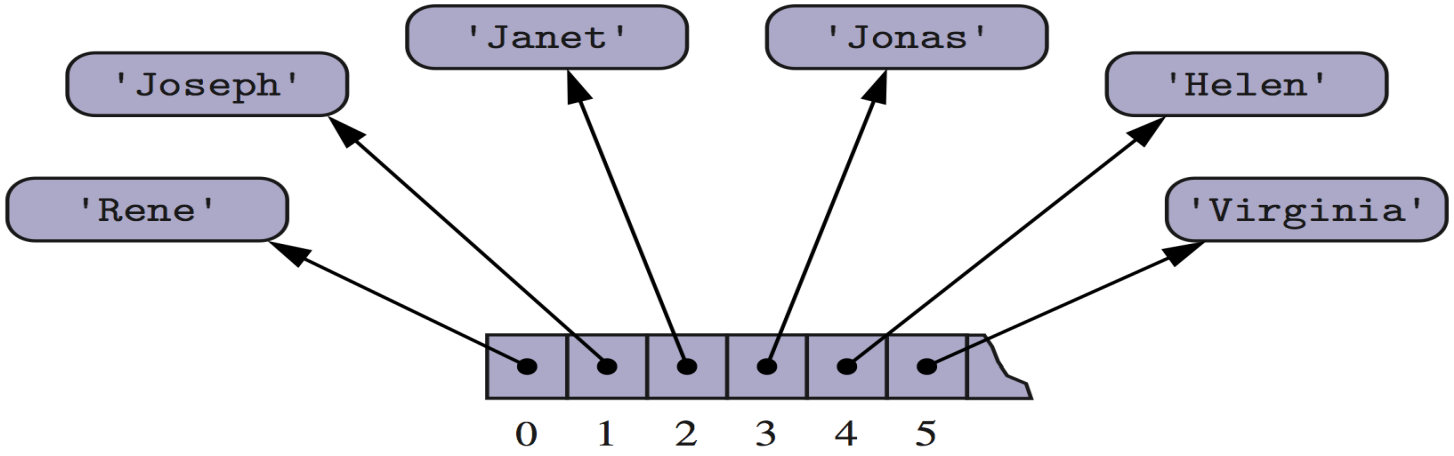
# Array

- An **array** is a data structure that stores a collection of elements of the *same data type* in *continuous memory* locations.

- An array provides:
  - a memory-efficient storage of a collection of elements of the same type
  - a fast direct-access to its individual elements (via indexing).
- Python:
  - **Does not** have an array as a built-in type, unlike e.g. Java and C/C++.
  - Implements the **str**, **tuple**, and **list** sequence data types (under the hood) using an array data structure, for example, the reference Python implementation uses the C programming language for this.

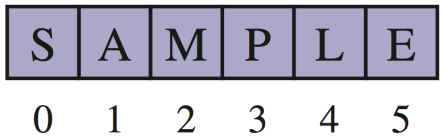
# Python Objects

- In Python, **all data values are objects**, and variables store **references** to objects.
  - Unlike some other languages (like Java or C++), Python ***does not*** have a distinct category of *primitive* data types stored by value.
- In Python, array-based sequences are thus implemented as arrays of references to objects



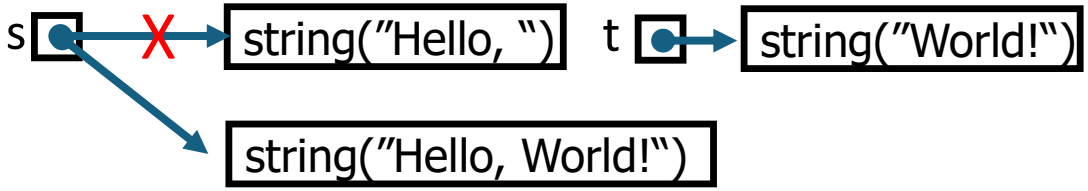
# Python Strings

- Python **string** is a fixed-size sequence objects of Unicode characters
  - For example, the layout of the string “SAMPLE” would be:



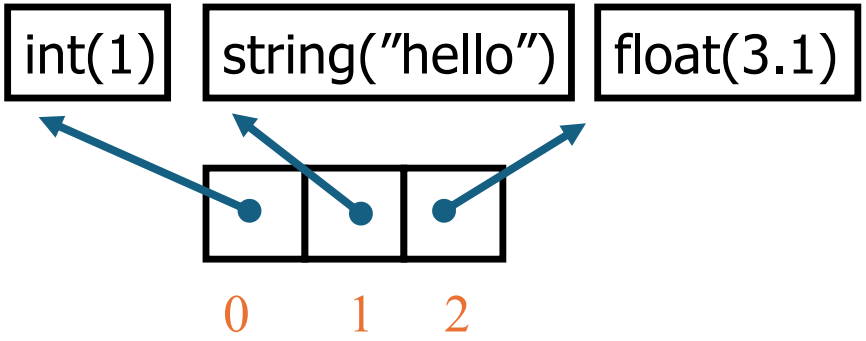

  - Uses a flexible representation (depending on the Unicode characters in the string), where all the characters in the string are either 1, 2, or 4 bytes.
    - Note, that a character is not a data type in Python, thus not object references!
  - Also, stores meta-data (not shown), e.g. the length of the string.
- Python strings are immutable, thus array content/length fixed

```
s = "Hello, "
t = " World!"
s = s + t          # Creates a new string
```



# Python Tuples

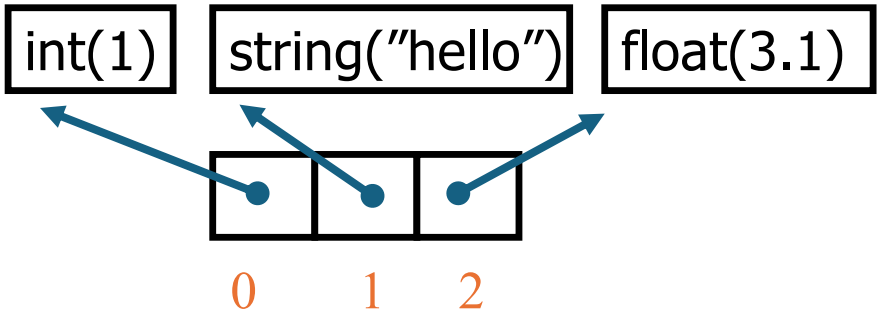
- Python **tuple** is a fixed-size sequence object references
  - For example, the layout of the tuple **(1, "hello", 3.1)** would be:



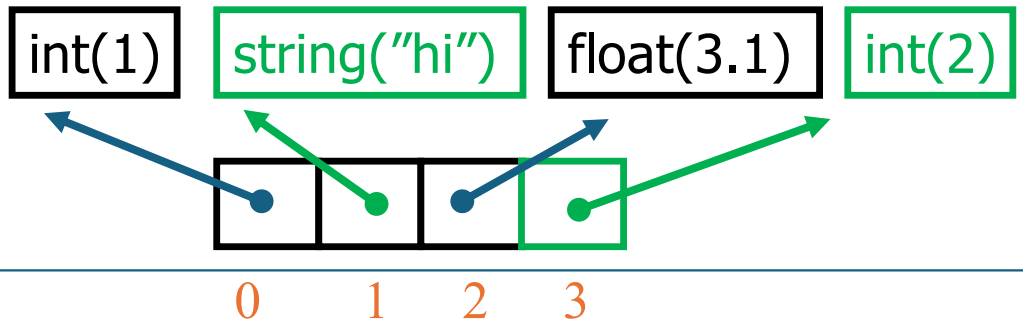

- Python tuples are immutable, thus array content/length is fixed
  - Note, if a tuple element is of a mutable type (e.g., a *list*) it can be changed, but change does not affect the array representing the tuple.

# Python Lists

- Python **list** is a sequence object references
  - For example, the layout of the list ***[ 1, "hello", 3.1 ]*** would be:



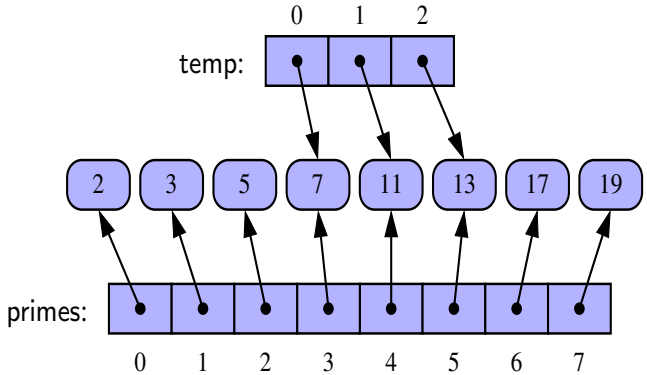

- Python lists are mutable, thus array content/length **may be changed**

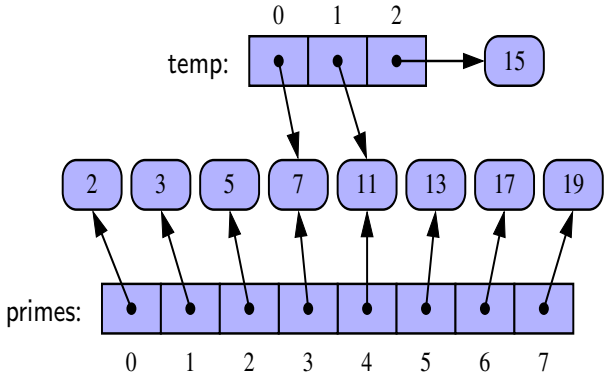



```
A = [1, "hello", 3.1]
A[1] = "hi"
t = " World!"
A.append(2)
```

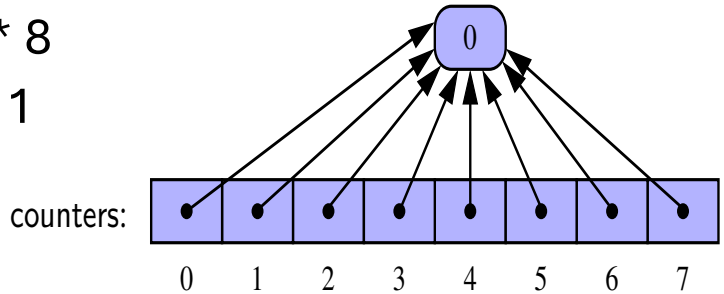
# Python Lists (continue)

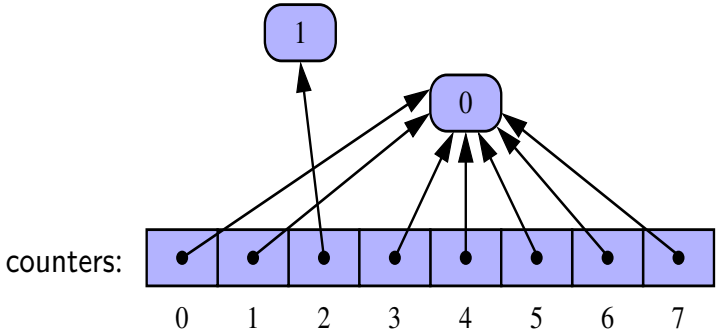
- Result of:
  - temp = primes[3:6]
  - temp[2] = 15



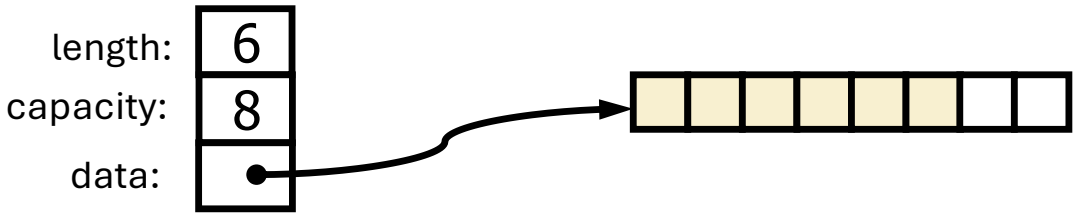

  - counters = [0] * 8
  - counters[2] += 1



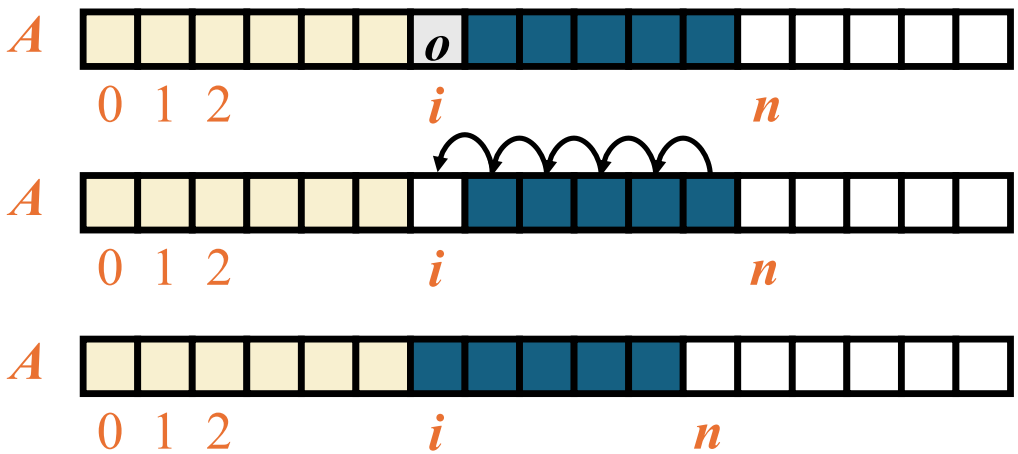
# Implementing a Dynamic Array

- Python ***list*** is implemented (under-the-hood) with a **dynamic array**
  - The size of a dynamic array may grow as we add elements to it
- How do we implement a dynamic array? Must store its:
  - length: Number of elements **currently in** the array
  - capacity: Number of elements it has hold (without resizing)
  - data: An fixed-size arrray (of length *capacity*) storing the elements



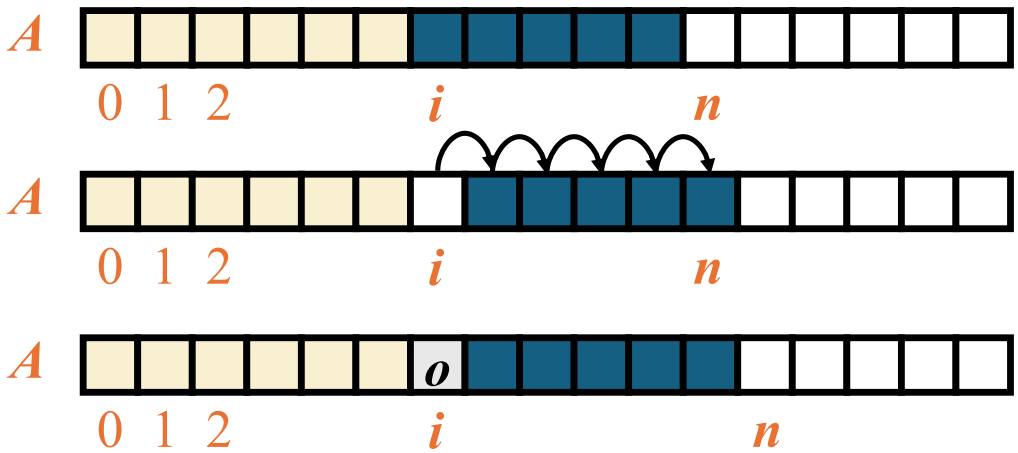
# Removal

- In an operation ***remove***($i$), remove element at index $\boldsymbol{i}$, we need to fill the hole left by the removed element by shifting backward the $\boldsymbol{n} - \boldsymbol{i} - 1$ elements $\boldsymbol{A}[\boldsymbol{i} + 1], \ldots, \boldsymbol{A}[\boldsymbol{n} - 1]$
- In the worst case ($\boldsymbol{i} = 0$), this takes $\boldsymbol{O}(\boldsymbol{n})$ time



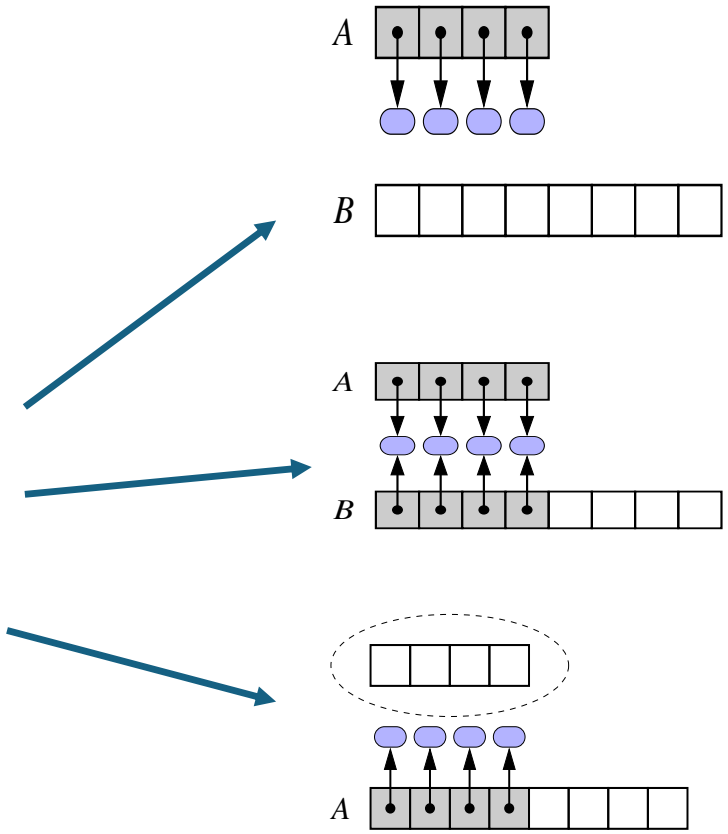
# Insertion

- In an operation $add(i, o)$, i.e. add element $o$ at index $i$, we need to make room for the new element by shifting forward the $n - i$ elements $A[i], \ldots, A[n-1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



# Insertion (when array is full) -> resize

- We first need  to resize the array before we can insert the new element
- Three step process:
  1) Create a larger array (B), e.g. double the capacity of the current one
  2) Copy elements in current array (A) to the new one (B)
  3) “Release” current array and make the new one the current



# Performance

- In an array-based implementation of a dynamic list:
  - The space used by the data structure is $O(n)$
  - Indexing the element at *i* takes $O(1)$ time
  - ***add*** and ***remove*** run in $O(n)$ time in worst/average case
    - Except when at the end of the list, then $O(1)$ (see though below)
- The ***add*** operation, when the array is full, instead of throwing an exception (as in fixed-sized arrays), we resize it.
  - Adding an element to the end of an array usually takes ***O(1)***, except, occasionally, when needing to resize, then ***O***(***n***)
  - We say the ***add*** operation er ***amortized O(1)***

# Efficiency of Python's Sequence Types (cont.)

| Operation | Running Time |
|---:|---|
| len(data) | $O(1)$ |
| data[j] | $O(1)$ |
| data.count(value) | $O(n)$ |
| data.index(value) | $O(k+1)$ |
| value **in** data | $O(k+1)$ |
| data1 == data2<br>(similarly !=, <, <=, >, >=) | $O(k+1)$ |
| data[j:k] | $O(k-j+1)$ |
| data1 + data2 | $O(n_1+n_2)$ |
| c * data | $O(cn)$ |

**Table 5.3:** Asymptotic performance of the nonmutating behaviors of the list and tuple classes. Identifiers data, data1, and data2 designate instances of the list or tuple class, and $n$, $n_1$, and $n_2$ their respective lengths. For the containment check and index method, $k$ represents the index of the leftmost occurrence (with $k = n$ if there is no occurrence). For comparisons between two sequences, we let $k$ denote the leftmost index at which they disagree or else $k = \min(n_1, n_2)$.

Figure from textbook

# Efficiency of Python's Sequence Types

| Operation | Running Time |
|---|---|
| data[j] = val | $O(1)$ |
| data.append(value) | $O(1)^*$ |
| data.insert(k, value) | $O(n-k+1)^*$ |
| data.pop( ) | $O(1)^*$ |
| data.pop(k)<br>**del** data[k] | $O(n-k)^*$ |
| data.remove(value) | $O(n)^*$ |
| data1.extend(data2)<br>data1 += data2 | $O(n_2)^*$ |
| data.reverse( ) | $O(n)$ |
| data.sort( ) | $O(n \log n)$ |

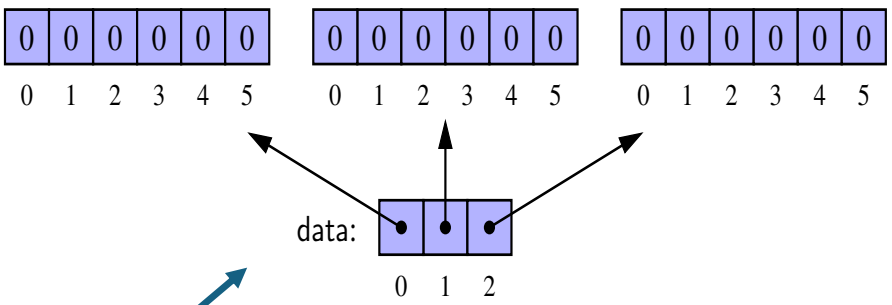
*amortized

**Table 5.4:** Asymptotic performance of the mutating behaviors of the list class. Identifiers data, data1, and data2 designate instances of the list class, and $n$, $n_1$, and $n_2$ their respective lengths.

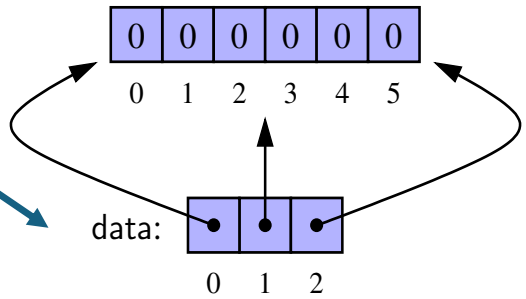
Figure from textbook

# Multi-Dimensional Data Sets

- Two-Dimentional datasets in Python are implement as a **list-of- lists**
- **Accessed as**
  - x = data[i][j]
  - print( data[1][2] )




- **Important to declare correctly:**
  - **Make an array of size 3 x 6**
  - data = [ [0] * 6 **for** j **in range**(3) ] # **Right**
  - data = [ [0] * 6 ] * 3 # **Wrong**



# Looking “under-the-hood”

Python 3.11
known limitations

```
1  data_wrong = [ [0] * 6 ] * 3
2  data_wrong[2][1] = 10
3  print(data_wrong[2][1], data_wrong[1][1])
4
5  data_right = [ [0] * 6 for j in range(3) ]
6  data_right[2][1] = 10
7  print(data_right[2][1], data_right[1][1])
```

Edit this code

line that just executed
next line to execute

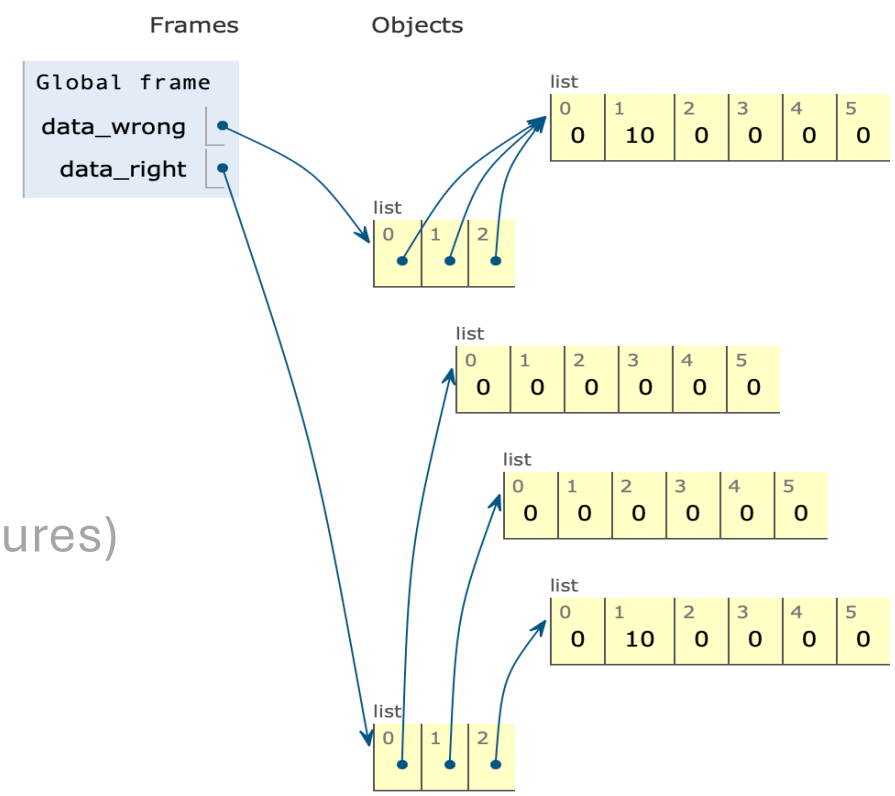
<< First  < Prev  Next >  Last >>

Done running (12 steps)

Move and hide objects (instructions)

Print output (drag lower right corner to resize)

```
10 10
10 0
```




**Use it to understand your data** (structures) **better:** https://pythontutor.com

Link to example: click here

# Summary

- The main takeaways are:
  - Arrays, both fixed-sized and dynamic-sized, are fundamental data structures, used on their own and to implement other data structures
  - For example, in Python:
    - Lists are implement “under-the-hood“ using a dynamic-sized array.
    - You should know the time-complexity **O( … )** of the most common list operations, and understand why the complexity is that way.
  - Understand how to use (and implement, if needed) both fixed- and dynamically-sized array data structures
  - **Understand Python‘s object reference model**

Image created using *Bing image creator*